Section 3: SOLID Principles for Interviews

This section is part of the 5-hour LLD course by Mukul Raina

- YouTube Course: https://www.youtube.com/watch?v=VSABVEBDhf0
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 25: Section Intro: SOLID Principles for Interviews

SLIDE 26: Single Responsibility Principle: What “One Reason to Change” Means

SLIDE 27: SRP Violation: The God UserService

SLIDE 27B: SRP Refactored: Clean Separation

SLIDE 28: How SRP Connects to Microservice Thinking

SLIDE 29: Open-Closed Principle: Extension Without Modification

SLIDE 30: OCP Before and After: Adding Discount Types

SLIDE 30B: Strategy Pattern Preview as OCP in Action

SLIDE 30C: Interview Practice: SRP and OCP

SLIDE 31: Liskov Substitution Principle: Behavioral Subtyping

SLIDE 32: The Rectangle-Square Problem Explained Properly

SLIDE 32B: Real LSP Violation: ReadOnlyRepository

SLIDE 33: Why LSP Violations Cause Production Bugs

SLIDE 33B: Interview Practice: LSP

SLIDE 34: Interface Segregation Principle: Fat vs Focused Interfaces

SLIDE 35: ISP Violation: The Machine Interface

SLIDE 35B: ISP Refactored into Focused Interfaces

SLIDE 36: Dependency Inversion Principle: Depending on Abstractions

SLIDE 37: DIP Worked Example: Order Service and Payment Interface

SLIDE 37B: How DIP Enables Testability

SLIDE 37C: Note It Down: SOLID at a Glance

SLIDE 38: SOLID Violations as Code Smells: Quick Recognition Guide

SLIDE 39: SOLID Interview Practice and Common Mistakes

SLIDE 40: Section Summary and Cheat Sheet

SLIDE 25: Section Intro: SOLID Principles for Interviews

Section 3: SOLID Principles for Interviews

SOLID is a set of five design principles that guide how you structure classes and their relationships. In LLD interviews, SOLID is not tested as theory. Interviewers evaluate whether you instinctively apply these principles when designing classes, and whether you can spot violations when they probe your design. The principles build directly on the OOP foundations from Section 2: SRP refines encapsulation, OCP formalizes polymorphism for extensibility, LSP constrains inheritance, ISP refines abstraction, and DIP enforces composition over concrete dependencies.

What We Will Cover

1. Single Responsibility Principle (SRP): what “one reason to change” actually means
2. Open-Closed Principle (OCP): designing for extension without modification
3. Liskov Substitution Principle (LSP): behavioral subtyping, not just type compatibility
4. Interface Segregation Principle (ISP): fat interfaces vs focused interfaces
5. Dependency Inversion Principle (DIP): depending on abstractions, not implementations
6. SOLID violations as code smells: a quick recognition guide for interviews

Where SOLID Appears in LLD Interviews

Interview Moment	What the Interviewer Is Testing
“How would you add a new feature?”	OCP: can you extend without modifying existing code?
“This class is doing too much”	SRP: can you decompose responsibilities?
“What if we swap this implementation?”	DIP: are your dependencies on abstractions?
“Does this subclass really belong here?”	LSP: does the subclass honor the parent’s contract?
“Your interface is too large”	ISP: can you break it into focused contracts?
Design review / code walkthrough	All five: can you spot violations in your own design?

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 26: Single Responsibility Principle: What “One Reason to Change” Means

The Single Responsibility Principle states that a class should have only one reason to change. This is the most frequently misunderstood SOLID principle because “one responsibility” sounds vague. The precise definition is about change drivers: a class should be affected by changes from exactly one source.

A change driver is a stakeholder, a business domain, or a technical concern that could independently require modifications to the class. If two unrelated changes (a new business rule for pricing AND a new format for log output) both require editing the same class, that class has more than one responsibility.

Concrete Example

Consider an `InvoiceService` that calculates totals, formats PDF output, and sends emails. Three different change drivers affect this single class:

1. The finance team changes the tax calculation rules
2. The design team changes the PDF invoice layout
3. The infrastructure team switches from SMTP to a third-party email API

Each of these changes comes from a different stakeholder and a different domain. Yet all three require editing the same `InvoiceService` class. This means a change requested by the design team (PDF layout) could accidentally break the tax calculation logic if the developer modifies the wrong method or introduces a merge conflict. SRP violations increase the blast radius of every change.

Interview Tip: When an interviewer asks "does this class have a single responsibility?", do not answer with "it does one thing." Instead, identify the distinct change drivers. "This class would need to change if the tax rules change OR if the PDF format changes OR if the email provider changes. That is three reasons to change, so it violates SRP."

Interview Relevance: SRP is the most commonly tested SOLID principle. Interviewers often present a class with 5-6 methods across different domains and ask you to refactor it.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 27: SRP Violation: The God UserService

Here is a `UserService` that handles authentication, profile management, email notifications, and audit logging. This is a realistic example of what appears in production codebases and in LLD interview questions.

```
public class UserService {
    private final UserRepository userRepository;
    private final EmailClient emailClient;
    private final AuditLogger auditLogger;

    public UserService(UserRepository userRepository, EmailCl
ient emailClient,
                       AuditLogger auditLogger) {
        this.userRepository = userRepository;
        this.emailClient = emailClient;
        this.auditLogger = auditLogger;
    }
```

```
    public boolean authenticate(String email, String passwor
d) {
        User user = userRepository.findByEmail(email);
        if (user == null || !PasswordHasher.verify(password,
user.getPasswordHash())) {
            auditLogger.log("FAILED_LOGIN", email);
            return false;
        }
        auditLogger.log("SUCCESSFUL_LOGIN", email);
        return true;
    }

    public void updateProfile(String userId, String newName,
String newBio) {
        User user = userRepository.findById(userId);
        user.setName(newName);
        user.setBio(newBio);
        userRepository.save(user);
        emailClient.send(user.getEmail(), "Profile Updated",
            "Your profile has been updated successfully.");
        auditLogger.log("PROFILE_UPDATED", userId);
    }

    public void resetPassword(String email) {
        User user = userRepository.findByEmail(email);
        String tempPassword = PasswordGenerator.generate();
        user.setPasswordHash(PasswordHasher.hash(tempPasswor
d));
        userRepository.save(user);
        emailClient.send(email, "Password Reset",
            "Your temporary password is: " + tempPassword);
        auditLogger.log("PASSWORD_RESET", email);
    }
}
```

Four Distinct Change Drivers in One Class

Change Driver	Example Change	Methods Affected
Authentication policy	Add two-factor authentication	`authenticate`, `resetPassword`
Profile business rules	Add profile picture upload, bio length limits	`updateProfile`
Notification requirements	Switch to SMS, add templates	`updateProfile`, `resetPassword`
Audit compliance	Change log format, add retention policy	All three methods

A change to the email template system could introduce a bug in authentication if a careless merge modifies shared state. Every method in this class is a minefield for unrelated changes.

Interview Relevance: If an interviewer presents you with a class like this and asks "what is wrong with this design?", the answer is SRP violation. The next slide shows the refactored version.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 27B: SRP Refactored: Clean Separation

Each responsibility from the `UserService` now lives in its own class with its own single change driver.

```
public class AuthenticationService {
    private final UserRepository userRepository;
    private final AuditService auditService;

    public AuthenticationService(UserRepository userRepositor
y,
                                 AuditService auditService) {
        this.userRepository = userRepository;
        this.auditService = auditService;
```

```
    }

    public AuthResult authenticate(String email, String passw
ord) {
        User user = userRepository.findByEmail(email);
        if (user == null || !PasswordHasher.verify(password,
user.getPasswordHash())) {
            auditService.record(AuditEvent.failedLogin(emai
l));
            return AuthResult.failure("Invalid credentials");
        }
        auditService.record(AuditEvent.successfulLogin(emai
l));
        return AuthResult.success(user.getId());
    }
}
```

```
public class ProfileService {
    private final UserRepository userRepository;
    private final NotificationService notificationService;

    public ProfileService(UserRepository userRepository,
                          NotificationService notificationSer
vice) {
        this.userRepository = userRepository;
        this.notificationService = notificationService;
    }

    public void updateProfile(String userId, ProfileUpdateReq
uest request) {
        User user = userRepository.findById(userId);
        user.updateName(request.getName());
        user.updateBio(request.getBio());
        userRepository.save(user);
        notificationService.sendProfileUpdateConfirmation(use
```

```
r);
    }
}
```

```
public class PasswordResetService {
    private final UserRepository userRepository;
    private final NotificationService notificationService;
    private final AuditService auditService;

    public void resetPassword(String email) {
        User user = userRepository.findByEmail(email);
        String tempPassword = PasswordGenerator.generate();
        user.changePassword(tempPassword);
        userRepository.save(user);
        notificationService.sendPasswordReset(user, tempPassw
ord);
        auditService.record(AuditEvent.passwordReset(email));
    }
}
```

Now a change to authentication policy affects only `AuthenticationService`. A change to notification delivery affects only `NotificationService`. A change to profile rules affects only `ProfileService`. The blast radius of every change is contained to a single class.

Interview Relevance: Presenting this refactored version demonstrates that you can decompose responsibilities, not just identify violations. Interviewers want to see both the diagnosis and the treatment.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 28: How SRP Connects to Microservice Thinking

SRP at the class level is the same principle that drives microservice decomposition at the system level. Each microservice owns a single business domain and changes independently. The class-level SRP practice you develop in LLD interviews is the foundation for the service-level decomposition you discuss in system design interviews.

From Classes to Services

SRP at Class Level	Same Principle at Service Level
`AuthenticationService` class	Authentication Microservice
`ProfileService` class	User Profile Microservice
`NotificationService` class	Notification Microservice
`AuditService` class	Audit/Logging Microservice
Each class has one change driver	Each service is owned by one team
Classes communicate through interfaces	Services communicate through APIs

The refactored `UserService` from the previous slide maps directly to four microservices, each independently deployable, independently testable, and owned by a separate team. This is not a coincidence. SRP and microservice boundaries are the same concept at different scales.

Where the Analogy Breaks Down

The cost of splitting is different. Splitting a class into three classes adds minimal overhead. Splitting a monolith into three services adds network latency, distributed transaction complexity, deployment infrastructure, and operational monitoring. This is why SRP does not mean “split everything.” Both at class level and service level, the question is whether the cost of coupling (shared change drivers) exceeds the cost of separation (additional coordination).

Interview Relevance: Making this connection between class-level SRP and service-level decomposition shows the interviewer that you think across abstraction levels, a signal of senior-level design thinking.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 29: Open-Closed Principle: Extension Without Modification

The Open-Closed Principle states that a class should be open for extension but closed for modification. In practical terms, you should be able to add new behavior to a system without changing existing, tested code.

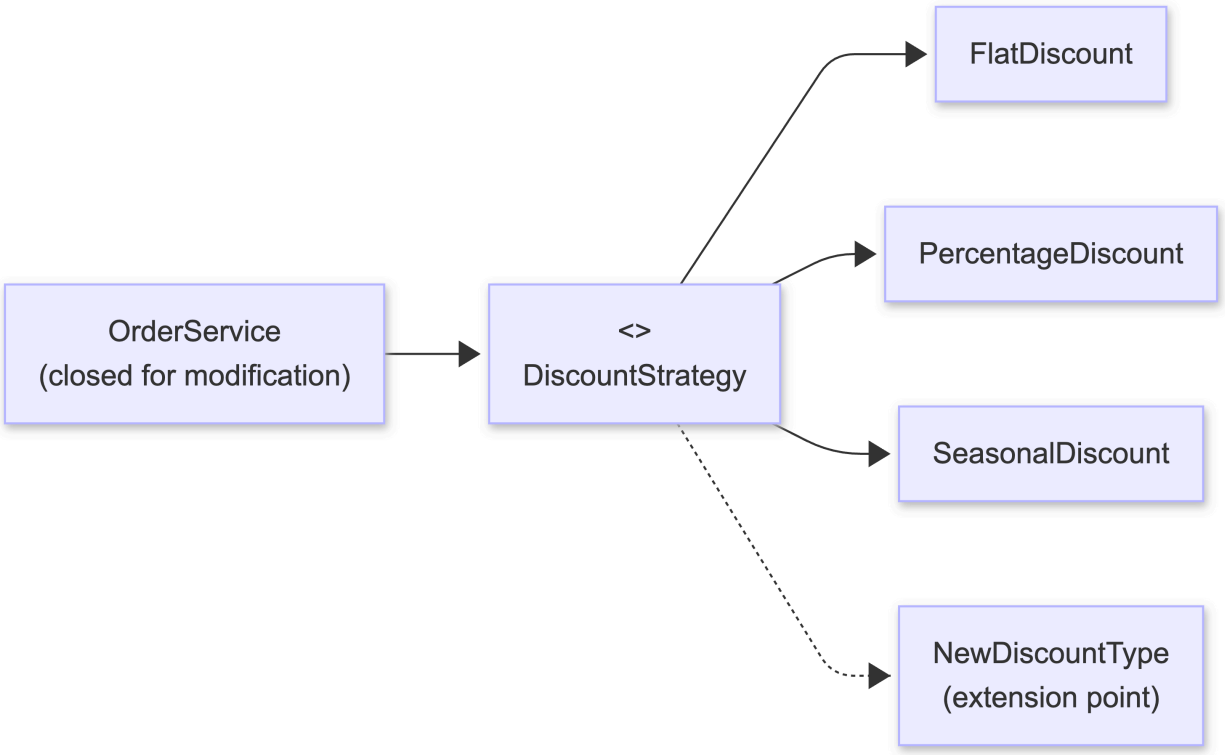
This does not mean you never modify existing classes. It means you design your classes so that the most likely type of change (adding a new variant, supporting a new format, handling a new case) can be done by writing new code rather than editing old code.

Why OCP Matters

Every time you modify an existing class, you risk breaking something that was already working. If adding a new discount type requires editing the `PriceCalculator`, and that class also handles tax calculation and currency conversion, a bug in the new discount logic could break tax calculations. OCP minimizes this risk by ensuring new behavior is added through new classes, not by editing existing ones.

The Connection to Polymorphism

OCP is the formal principle behind the polymorphic design from Section 2. When you define an interface and add new implementations, you are following OCP. The interface is “closed” (its contract does not change). The system is “open” (new implementations can be added freely).



The dashed line to `NewDiscountType` represents the extension point. Adding it requires zero changes to `OrderService` or any existing discount class.

Interview Relevance: OCP is the second most tested SOLID principle after SRP. Interviewers test it by asking “how would you add a new type of X?” If your answer involves modifying existing code, you have violated OCP.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 30: OCP Before and After: Adding Discount Types

Before OCP: The Switch Statement Approach

```
public class PriceCalculator {
    public Money calculateDiscount(Order order, String discou
ntType) {
        switch (discountType) {
            case "FLAT":
```

```
                return Money.of(10.00);
            case "PERCENTAGE":
                return order.getSubtotal().multiplyBy(0.15);
            case "BUY_ONE_GET_ONE":
                return order.getCheapestItemPrice();
            default:
                return Money.zero();
        }
    }
}
```

Every new discount type requires adding a new case to this switch statement. The class is modified for every new business requirement. If the marketing team creates promotions frequently, this class is edited constantly, and every edit risks breaking existing discount logic.

After OCP: The Interface Approach

```
public interface DiscountStrategy {
    Money calculateDiscount(Order order);
    boolean isApplicable(Order order);
}
```

```
public class FlatDiscount implements DiscountStrategy {
    private final Money discountAmount;

    public FlatDiscount(Money discountAmount) {
        this.discountAmount = discountAmount;
    }

    @Override
    public Money calculateDiscount(Order order) {
        return discountAmount;
    }

    @Override
```

```
    public boolean isApplicable(Order order) {
        return order.getSubtotal().isGreaterThan(Money.of(50.
00));
    }
}
```

```
public class PriceCalculator {
    private final List<DiscountStrategy> availableDiscounts;

    public PriceCalculator(List<DiscountStrategy> availableDi
scounts) {
        this.availableDiscounts = availableDiscounts;
    }

    public Money calculateBestDiscount(Order order) {
        return availableDiscounts.stream()
            .filter(discount -> discount.isApplicable(order))
            .map(discount -> discount.calculateDiscount(orde
r))
            .max(Comparator.naturalOrder())
            .orElse(Money.zero());
    }
}
```

Now `PriceCalculator` is closed for modification. Adding a “Seasonal 20% Off” discount means writing a `SeasonalDiscount` class and registering it. The `PriceCalculator` class is never edited.

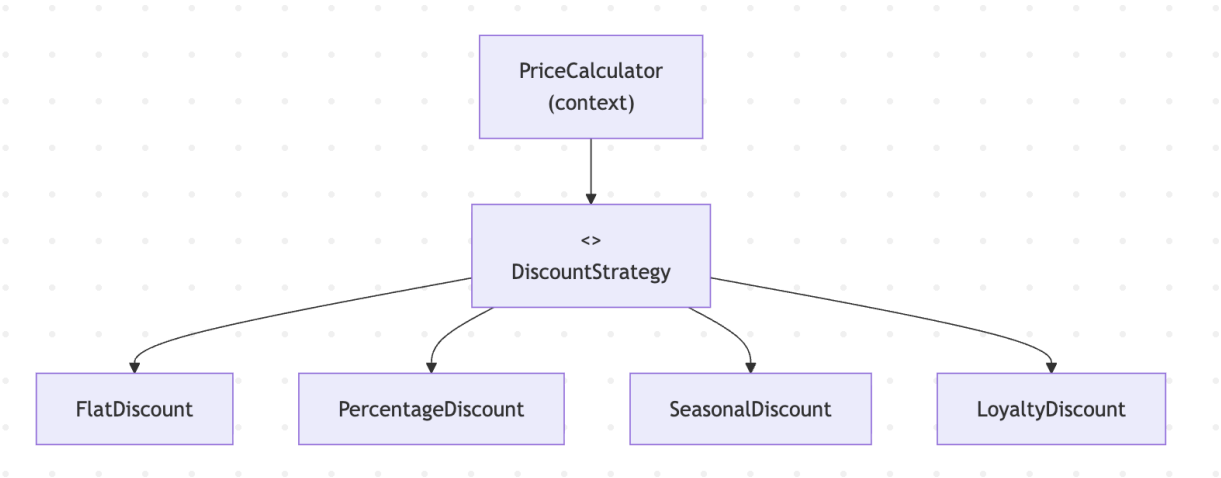
Interview Relevance: This before/after transformation is the exact pattern interviewers expect when they ask “how would you make this extensible?” Show the switch statement first, explain why it violates OCP, then show the interface-based solution.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

SLIDE 30B: Strategy Pattern Preview as OCP in Action

The discount example from the previous slide is actually the Strategy pattern in disguise. The Strategy pattern is the most direct implementation of OCP: define a family of algorithms behind an interface, and let the calling code work with any of them interchangeably.

The Structure



The `PriceCalculator` is the context. It holds a reference to the strategy interface and delegates the discount calculation. Each concrete strategy encapsulates one algorithm. We will cover the Strategy pattern in full depth in Section 6 (Behavioral Design Patterns), but recognizing this connection now is important: OCP tells you why to design this way, and Strategy tells you how.

Why This Matters Beyond Discounts

The same OCP + Strategy structure applies to any requirement that says "support multiple types of X":

Requirement	Strategy Interface	Implementations
Multiple discount types	`DiscountStrategy`	Flat, Percentage, Seasonal, Loyalty
Multiple payment methods	`PaymentProcessor`	Stripe, PayPal, BankTransfer
Multiple export formats	`DocumentExporter`	PDF, DOCX, HTML, CSV
Multiple notification channels	`NotificationChannel`	Email, SMS, Push, Webhook

Requirement	Strategy Interface	Implementations
Multiple compression algorithms	`CompressionStrategy`	Gzip, Snappy, LZ4, Zstd

Every row in this table is a place where OCP prevents you from writing a growing switch statement.

Interview Tip: When you hear "support multiple types of X" in an LLD interview, your first instinct should be to define an interface for X. Say this out loud to the interviewer: "Since we need multiple types of X, I will define an interface so we can add new types without modifying existing code." This demonstrates OCP thinking without needing to name the principle explicitly.

Interview Relevance: The connection between OCP and Strategy is one of the highest-value insights for LLD interviews. Interviewers expect you to reach for this pattern whenever you hear "multiple types of."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 30C: Interview Practice: SRP and OCP

Scenario

You are designing an order processing system for a platform like Shopify. The `OrderProcessor` class currently handles order validation, inventory checking, payment processing, shipping label generation, and sending confirmation emails. The interviewer asks: "What is wrong with this design, and how would you fix it?"

Pause the video and identify the problems and your refactored design for 90 seconds before continuing.

Weak Answer

"The class is too big. I would split it into smaller classes."

This answer identifies the symptom but not the principle. It does not explain why the class should be split or how to decide where the boundaries should be.

Strong Answer

“This class violates SRP because it has five distinct change drivers: validation rules, inventory management, payment processing, shipping integration, and notification delivery. A change to the shipping provider should not require editing the same class that handles payment processing.

I would refactor into five classes, each behind an interface: `OrderValidator`, `InventoryService`, `PaymentProcessor`, `ShippingService`, and `NotificationService`. The `OrderProcessor` becomes an orchestrator that delegates to each service through its interface.

This also enables OCP: when we add a new payment provider, we write a new `PaymentProcessor` implementation. When we add a new notification channel, we write a new `NotificationService` implementation. The orchestrator never changes.”

The strong answer names the principles, identifies specific change drivers, and explains how the refactored design enables future extension.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 31: Liskov Substitution Principle: Behavioral Subtyping

The Liskov Substitution Principle states that if class B is a subtype of class A, you should be able to replace A with B without the program’s behavior becoming incorrect. This is not just about type compatibility (will it compile). It is about behavioral compatibility (will it work correctly at runtime).

What “Behavioral Compatibility” Means

A subtype must honor four constraints from the parent type:

1. **Preconditions cannot be strengthened.** If the parent accepts any positive number, the subtype cannot restrict to only even numbers.
2. **Postconditions cannot be weakened.** If the parent guarantees a non-null return value, the subtype cannot return null.
3. **Invariants must be preserved.** If the parent guarantees that balance >= 0, the subtype must maintain that invariant.

4. **No unexpected side effects.** If the parent's `save()` method saves to the database, the subtype's `save()` should not also send an email as a surprise.

When any of these constraints is violated, code that works correctly with the parent type will break when given the subtype. The calling code has no way to know it is broken because the type system says the substitution is valid.

Interview Tip: LSP is the least intuitive SOLID principle. When an interviewer brings it up, they are testing whether you understand the difference between type-level compatibility (compiler says it is fine) and behavioral compatibility (the program actually works correctly). Focus your answer on the behavioral contract, not on Java's type system.

Interview Relevance: LSP violations are tested through concrete examples (Rectangle-Square, read-only collections, exception-throwing overrides). Know at least two examples and be able to explain why the substitution breaks.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 32: The Rectangle-Square Problem Explained Properly

This is the most famous LSP example. Mathematically, a square is a rectangle. But in object-oriented design, making `Square` extend `Rectangle` creates a behavioral violation.

```
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }
```

```
    public int area() {
        return width * height;
    }
}
```

```
public class Square extends Rectangle {
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}
```

Where the Substitution Breaks

```
public void resizeAndVerify(Rectangle rectangle) {
    rectangle.setWidth(5);
    rectangle.setHeight(10);
    assert rectangle.area() == 50;
}
```

This method works correctly for any `Rectangle`. Set width to 5, set height to 10, area is 50. But pass a `Square`, and `setHeight(10)` also sets the width to 10. The area becomes 100, not 50. The assertion fails. The program's behavior is incorrect even though the type system accepted `Square` as a `Rectangle`.

Why This Happens

`Rectangle` has an implicit postcondition: setting the height does not change the width. `Square` violates this postcondition. The calling code relies on the postcondition (it expects width to remain 5 after setting height), so the substitution breaks.

The Fix

Make `Rectangle` and `Square` separate classes that both implement a `Shape` interface. Neither extends the other. The "is-a" relationship from mathematics does not translate to a valid "is-a" relationship in OOP because the behavioral contracts differ.

Interview Relevance: This example appears in interviews frequently. The key insight to communicate is: "Mathematical is-a and OOP is-a are not the same thing. OOP is-a requires behavioral compatibility, not just logical categorization."

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 32B: Real LSP Violation: ReadOnlyRepository

The Rectangle-Square problem feels academic. Here is an LSP violation that appears in real production systems and in LLD interviews.

```
public interface Repository<T> {
    T findById(String id);
    List<T> findAll();
    void save(T entity);
    void delete(String id);
}
```

```
public class ReadOnlyRepository<T> implements Repository<T> {
    private final DataSource dataSource;

    @Override
    public T findById(String id) {
        return dataSource.query(id);
```

```
    }

    @Override
    public List<T> findAll() {
        return dataSource.queryAll();
    }

    @Override
    public void save(T entity) {
        throw new UnsupportedOperationException("This reposit
ory is read-only");
    }

    @Override
    public void delete(String id) {
        throw new UnsupportedOperationException("This reposit
ory is read-only");
    }
}
```

Why This Violates LSP

Any code that accepts a `Repository<T>` reasonably expects that `save()` and `delete()` work. The `ReadOnlyRepository` compiles fine and passes type checks, but it throws exceptions for operations that the interface promises to support. Code that loops through a list of repositories and calls `save()` on each will crash at runtime when it hits the read-only one. The type system lied.

The Fix: Interface Segregation

```
public interface ReadableRepository<T> {
    T findById(String id);
    List<T> findAll();
}

public interface WritableRepository<T> extends ReadableReposi
tory<T> {
```

```
    void save(T entity);
    void delete(String id);
}
```

Now `ReadOnlyRepository` implements `ReadableRepository` and genuinely supports every method it declares. Code that needs to write uses `WritableRepository`. Code that only reads uses `ReadableRepository`. No surprises, no exceptions, no LSP violation.

Notice how fixing the LSP violation naturally led us to ISP (Interface Segregation Principle). The principles reinforce each other.

Interview Relevance: Interviewers use the "exception-throwing override" pattern to test LSP understanding. If you see a subclass that throws `UnsupportedOperationException` for a parent's method, flag it as an LSP violation and suggest splitting the interface.

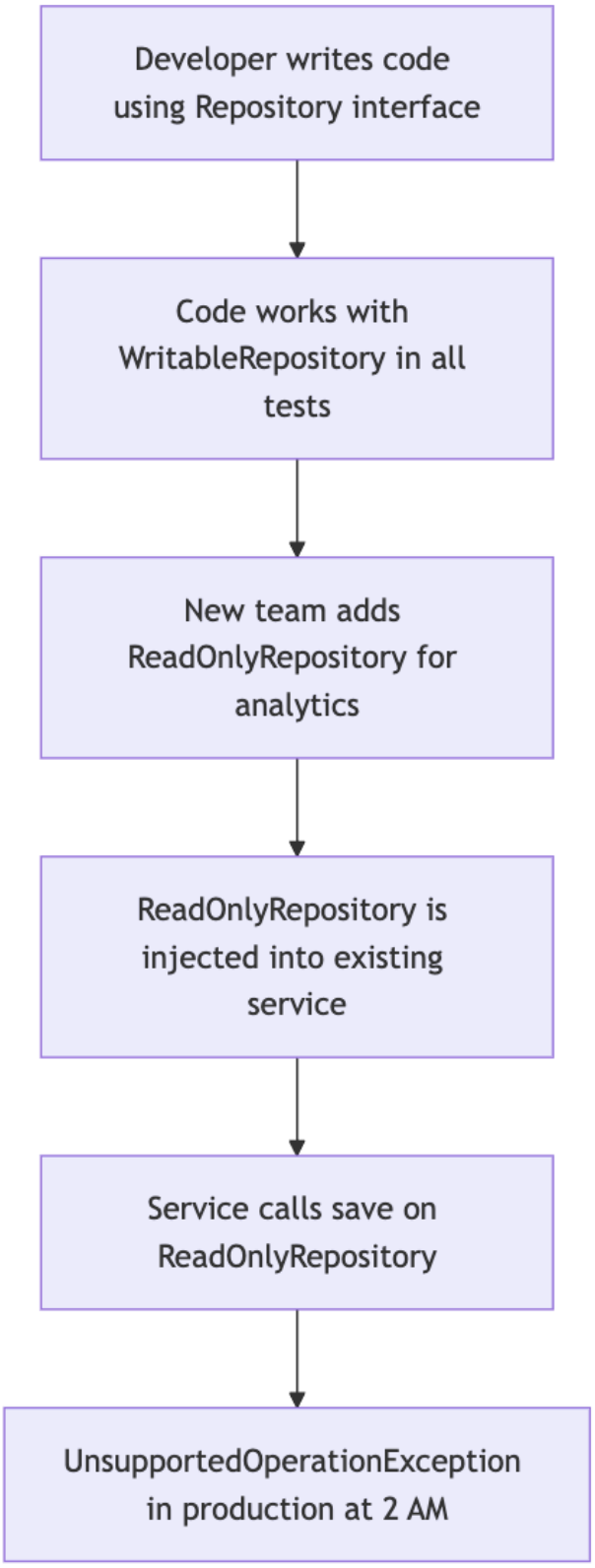
Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 33: Why LSP Violations Cause Production Bugs

LSP violations are dangerous because they pass all compile-time checks. The type system says the substitution is valid. The code compiles, the tests that use the parent type pass, and the code review looks clean. The bug only appears at runtime, often in a code path that was not tested with the specific subtype.

How LSP Violations Reach Production


Developer writes code using Repository interface
Code works with WritableRepository in all tests
New team adds ReadOnlyRepository for analytics
ReadOnlyRepository is injected into existing service
Service calls save on ReadOnlyRepository
UnsupportedOperationException in production at 2 AM

The developer who wrote the original service never anticipated that a `Repository` might not support `save()`. The developer who created `ReadOnlyRepository` never tested it in the context where `save()` is called. Each developer's code is correct in isolation. The bug exists in the gap between the interface contract and the implementation behavior.

Three Warning Signs of LSP Violations

1. A subclass overrides a method to throw an exception
2. A subclass overrides a method to do nothing (empty body)
3. Calling code uses `instanceof` checks before calling a method on the interface type

The third sign is particularly telling. If you see `if (repo instanceof WritableRepository)` before calling `save()`, it means the interface hierarchy is wrong. The type system should prevent invalid calls, not the programmer's vigilance.

Interview Relevance: Being able to explain why LSP violations are dangerous (not just what they are) separates candidates who understand SOLID deeply from those who memorized definitions.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 33B: Interview Practice: LSP

Scenario

You are reviewing a class hierarchy for a file storage system. The base class is `FileStorage` with methods `upload(file)`, `download(fileId)`, and `delete(fileId)`. A teammate has created `ArchiveStorage extends FileStorage` where `delete()` throws an `UnsupportedOperationException` because archived files must be retained for compliance.

The interviewer asks: "Is this a good design? What would you change?"

Pause the video and formulate your answer for 60 seconds before continuing.

Weak Answer

“It is fine because archived files should not be deleted. The exception tells callers not to delete.”

This answer misses the LSP violation entirely. The calling code does not know it is working with `ArchiveStorage`. A batch cleanup job that calls `delete()` on all `FileStorage` instances will crash on the archive.

Strong Answer

“This violates the Liskov Substitution Principle. Any code that accepts `FileStorage` expects `delete()` to work. `ArchiveStorage` breaks that contract by throwing an exception. The fix is to split the interface:

A `ReadableStorage` interface with `download()`. A `DeletableStorage` interface that extends `ReadableStorage` and adds `upload()` and `delete()`. `ArchiveStorage` implements only `ReadableStorage`. Batch cleanup jobs accept `DeletableStorage`, so they can never be passed an `ArchiveStorage` by mistake. The type system prevents the bug instead of relying on runtime exceptions.”

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 34: Interface Segregation Principle: Fat vs Focused Interfaces

The Interface Segregation Principle states that no class should be forced to implement methods it does not use. A “fat” interface is one that bundles many capabilities into a single contract, forcing implementors to provide methods that are irrelevant to their purpose.

ISP is closely related to LSP. When a class is forced to implement methods it does not need, the most common response is to throw `UnsupportedOperationException` or leave the method body empty. Both of these are LSP violations. ISP prevents LSP violations by ensuring that interfaces are small enough that every implementor can genuinely fulfill the entire contract.

The Problem with Fat Interfaces

A fat interface creates a false promise. It tells calling code: “any object of this type supports all of these operations.” When an implementor cannot actually support

all operations, the calling code has been lied to.

Fat interfaces also create unnecessary recompilation and redeployment. If you add a method to a 15-method interface, every implementing class must be updated, even the ones that have nothing to do with the new method.

The Goal

Design interfaces that are focused on a single capability or a closely related group of capabilities. Each interface should represent a role that an implementor plays, not a collection of everything the system might ever need.

Interview Relevance: ISP violations show up as large interfaces with 10+ methods where most implementors only care about 2-3. When you spot this in an interview, suggest splitting the interface by capability.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 35: ISP Violation: The Machine Interface

Consider an office equipment management system. The team defines a single interface for all machines.

```
public interface OfficeMachine {
    void print(Document document);
    void scan(Document document);
    void fax(Document document);
    void staple(Document document);
    void copy(Document document, int numberOfCopies);
}
```

Implementing a Basic Printer

```
public class BasicPrinter implements OfficeMachine {
    @Override
    public void print(Document document) {
        PrinterDriver.send(document.getContent());
```

```
    }

    @Override
    public void scan(Document document) {
        throw new UnsupportedOperationException("Basic printe
r cannot scan");
    }

    @Override
    public void fax(Document document) {
        throw new UnsupportedOperationException("Basic printe
r cannot fax");
    }

    @Override
    public void staple(Document document) {
        throw new UnsupportedOperationException("Basic printe
r cannot staple");
    }

    @Override
    public void copy(Document document, int numberOfCopies) {
        throw new UnsupportedOperationException("Basic printe
r cannot copy");
    }
}
```

The `BasicPrinter` genuinely supports one out of five methods. The other four are dead code that throws exceptions. This is both an ISP violation (forced to implement irrelevant methods) and an LSP violation (calling code expecting `OfficeMachine` gets exceptions on legitimate method calls).

Impact on the Codebase

When a new capability is added to `OfficeMachine` (for example, `void emailDocument(Document document, String recipient)`), every implementor must be updated, even `BasicPrinter`, which will just add another `throw new`

`UnsupportedOperationException()`. The interface becomes a growing tax on every implementor.

Interview Relevance: The “machine interface” is the canonical ISP example. Know it, but also be ready to show the refactored version with focused interfaces (next slide).

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 35B: ISP Refactored into Focused Interfaces

Split the fat `OfficeMachine` interface into focused interfaces, each representing a single capability.

```
public interface Printer {
    void print(Document document);
}

public interface Scanner {
    Document scan(String documentName);
}

public interface FaxMachine {
    void fax(Document document, String faxNumber);
}

public interface Copier {
    void copy(Document document, int numberOfCopies);
}
```

Implementing Machines with the Right Capabilities

```
public class BasicPrinter implements Printer {
    @Override
    public void print(Document document) {
```

```
        PrinterDriver.send(document.getContent());
    }
}

public class MultiFunctionDevice implements Printer, Scanner,
Copier {
    @Override
    public void print(Document document) {
        PrinterDriver.send(document.getContent());
    }

    @Override
    public Document scan(String documentName) {
        return ScannerDriver.capture(documentName);
    }

    @Override
    public void copy(Document document, int numberOfCopies) {
        Document scanned = scan(document.getName());
        for (int i = 0; i < numberOfCopies; i++) {
            print(scanned);
        }
    }
}
```

Now `BasicPrinter` implements only `Printer` and provides a real implementation for every method it declares. `MultiFunctionDevice` implements three interfaces and genuinely supports all three capabilities. No exceptions are thrown, no empty method bodies, no LSP violations.

Calling Code Depends on What It Needs

```
public class PrintService {
    private final Printer printer;

    public PrintService(Printer printer) {
```

```
        this.printer = printer;
    }
}

public class DocumentArchiver {
    private final Scanner scanner;

    public DocumentArchiver(Scanner scanner) {
        this.scanner = scanner;
    }
}
```

`PrintService` depends only on `Printer`. It accepts a `BasicPrinter`, a `MultiFunctionDevice`, or any future class that can print. `DocumentArchiver` depends only on `Scanner`. Each service declares exactly what capability it needs, nothing more.

Interview Tip: When you define interfaces in your LLD answer, aim for 2-4 methods per interface. If you find yourself defining an interface with 8+ methods, ask yourself: "Are these all part of the same capability, or am I bundling unrelated operations?" If they are unrelated, split the interface.

Interview Relevance: ISP is tested by asking you to design interfaces for a system with different device types, user roles, or service tiers. The interviewer wants to see focused interfaces, not a single God interface.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 36: Dependency Inversion Principle: Depending on Abstractions

The Dependency Inversion Principle has two parts:

1. High-level modules should not depend on low-level modules. Both should depend on abstractions.
2. Abstractions should not depend on details. Details should depend on abstractions.

In simpler terms: your business logic should never directly reference a specific technology, library, or infrastructure component. Instead, define an interface that represents what you need, and let the concrete implementation satisfy that interface.

Without DIP: Direct Dependency

```
public class OrderService {
    private final StripePaymentClient stripeClient;
    private final SendGridEmailClient sendGridClient;
    private final PostgresOrderRepository orderRepository;

    public OrderService() {
        this.stripeClient = new StripePaymentClient("sk_live_
abc123");
        this.sendGridClient = new SendGridEmailClient("sg_key
_xyz");
        this.orderRepository = new PostgresOrderRepository("j
dbc:postgresql://prod:5432/orders");
    }
}
```

`OrderService` is hardwired to Stripe, SendGrid, and PostgreSQL. Switching to PayPal requires editing `OrderService`. Writing a unit test requires a live Stripe account and a running PostgreSQL database. The business logic (processing orders) is entangled with infrastructure details.

With DIP: Depending on Abstractions

```
public class OrderService {
    private final PaymentProcessor paymentProcessor;
    private final NotificationSender notificationSender;
    private final OrderRepository orderRepository;

    public OrderService(PaymentProcessor paymentProcessor,
                        NotificationSender notificationSende
r,
```

```
                            OrderRepository orderRepository) {
        this.paymentProcessor = paymentProcessor;
        this.notificationSender = notificationSender;
        this.orderRepository = orderRepository;
    }
}
```

`OrderService` depends on three interfaces. It does not know or care whether payments go through Stripe or PayPal, whether notifications go through SendGrid or Twilio, or whether orders are stored in PostgreSQL or MongoDB. The concrete implementations are injected from outside.

Interview Relevance: DIP is tested every time an interviewer looks at your class diagram's dependency arrows. If arrows point from business logic classes toward infrastructure classes, you have violated DIP. Arrows should point toward abstractions.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 37: DIP Worked Example: Order Service and Payment Interface

Here is the full DIP-compliant design for an order processing system on a platform like Shopify.

The Abstractions

```
public interface PaymentProcessor {
    PaymentResult charge(String customerId, Money amount);
    PaymentResult refund(String transactionId);
}

public interface OrderRepository {
    void save(Order order);
    Optional<Order> findById(String orderId);
```

```
    List<Order> findByCustomer(String customerId);
}
```

The Business Logic (High-Level Module)

```
public class OrderService {
    private final PaymentProcessor paymentProcessor;
    private final OrderRepository orderRepository;

    public OrderService(PaymentProcessor paymentProcessor,
                        OrderRepository orderRepository) {
        this.paymentProcessor = paymentProcessor;
        this.orderRepository = orderRepository;
    }

    public OrderResult placeOrder(String customerId, List<Ord
erItem> items) {
        Money total = calculateTotal(items);
        PaymentResult paymentResult = paymentProcessor.charge
(customerId, total);

        if (!paymentResult.isSuccessful()) {
            return OrderResult.paymentFailed(paymentResult.ge
tErrorMessage());
        }

        Order order = new Order(customerId, items, total, pay
mentResult.getTransactionId());
        orderRepository.save(order);
        return OrderResult.success(order.getId());
    }
}
```

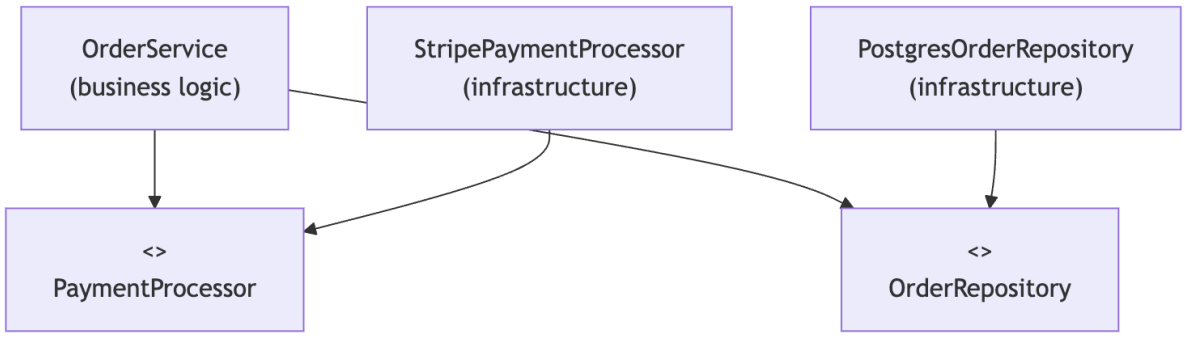
The Infrastructure (Low-Level Modules)

```
public class StripePaymentProcessor implements PaymentProcess
or {
    private final StripeClient stripeClient;

    public StripePaymentProcessor(StripeClient stripeClient)
{
        this.stripeClient = stripeClient;
    }

    @Override
    public PaymentResult charge(String customerId, Money amou
nt) {
        StripeCharge charge = stripeClient.createCharge(
            amount.toCents(), customerId
        );
        return new PaymentResult(charge.getId(), charge.getSt
atus().equals("succeeded"));
    }
}
```

The Dependency Direction



Both the high-level module (`OrderService`) and the low-level modules (`StripePaymentProcessor`, `PostgresOrderRepository`) depend on the abstractions. The abstractions own the dependency direction.

Interview Relevance: Drawing this diagram in an LLD interview, with dependency arrows pointing toward interfaces rather than toward concrete classes, is one of the clearest signals of design maturity an interviewer can observe.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 37B: How DIP Enables Testability

DIP's most immediate practical benefit is testability. When business logic depends on interfaces, you can substitute mock implementations in tests without needing real databases, real payment APIs, or real email servers.

Testing OrderService Without Infrastructure

```
public class OrderServiceTest {
    private OrderService orderService;
    private FakePaymentProcessor fakePaymentProcessor;
    private InMemoryOrderRepository inMemoryOrderRepository;

    public void setUp() {
        fakePaymentProcessor = new FakePaymentProcessor();
        inMemoryOrderRepository = new InMemoryOrderRepository
();
        orderService = new OrderService(fakePaymentProcessor,
inMemoryOrderRepository);
    }

    public void testSuccessfulOrder() {
        fakePaymentProcessor.setNextResult(PaymentResult.succ
ess("txn_001"));

        List<OrderItem> items = List.of(new OrderItem("widge
t", Money.of(25.00), 2));
        OrderResult result = orderService.placeOrder("custome
r_123", items);
```

```
        assert result.isSuccessful();
        assert inMemoryOrderRepository.findById(result.getOrderId()).isPresent();
    }

    public void testPaymentFailure() {
        fakePaymentProcessor.setNextResult(PaymentResult.failure("Card declined"));

        List<OrderItem> items = List.of(new OrderItem("widget", Money.of(25.00), 2));
        OrderResult result = orderService.placeOrder("customer_123", items);

        assert !result.isSuccessful();
        assert inMemoryOrderRepository.count() == 0;
    }
}
```

What DIP Made Possible

Without DIP	With DIP
Tests need a running PostgreSQL instance	Tests use `InMemoryOrderRepository`
Tests need valid Stripe API keys	Tests use `FakePaymentProcessor`
Tests take seconds (network calls)	Tests take milliseconds (in-memory)
Flaky tests from network timeouts	Deterministic, repeatable tests
Cannot test payment failure scenarios easily	Trivial: `setNextResult(failure)`

How This Shows Up in Interviews

When an interviewer asks “how would you test this?”, they are often indirectly testing DIP. If your answer requires setting up real infrastructure for a unit test, your design has DIP violations. If your answer is “I would inject a mock implementation of the interface,” you have demonstrated both DIP compliance and practical testing knowledge.

Interview Relevance: “How would you test this?” is one of the most common follow-up questions in LLD interviews. DIP-compliant designs have an immediate, clean answer. DIP-violating designs require awkward workarounds.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 37C: Note It Down: SOLID at a Glance

These are the five SOLID principles distilled to their essential test for violation. Write them down or screenshot this slide.

The One-Line Test for Each Principle

1. **SRP:** “How many reasons does this class have to change?” If more than one, split it.
2. **OCP:** “To add a new type of X, do I modify existing code or write a new class?” If modify, extract an interface.
3. **LSP:** “Can I substitute a subtype without the program breaking?” If not, the hierarchy is wrong.
4. **ISP:** “Does any implementor throw UnsupportedOperationException or leave methods empty?” If yes, split the interface.
5. **DIP:** “Does my business logic class import a concrete infrastructure class?” If yes, introduce an abstraction.

SOLID Violation Quick-Detection Table

Code Smell	Likely Violation	Fix
Class with 10+ methods across different domains	SRP	Split into focused service classes
Switch/if-else on type to decide behavior	OCP	Extract interface, one implementation per type
Subclass overrides method to throw exception	LSP + ISP	Split interface, adjust hierarchy

Code Smell	Likely Violation	Fix
Subclass overrides method with empty body	LSP	Remove inheritance, use composition
Interface with 8+ methods	ISP	Split into focused role-based interfaces
Constructor creates its own dependencies with `new`	DIP	Accept dependencies through constructor parameters
Business class imports database driver or HTTP client	DIP	Depend on an interface, inject implementation

Interview Tip: If you can recognize these seven code smells during an LLD interview and name the corresponding SOLID violation, you will demonstrate the pattern recognition that interviewers associate with senior-level candidates. Practice scanning code for these smells until the recognition is automatic.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 38: SOLID Violations as Code Smells: Quick Recognition Guide

This slide maps the most common code smells you will encounter in LLD interviews to the SOLID principle they violate. Each smell has a specific refactoring pattern. In an interview, recognizing the smell and naming the fix in under 30 seconds is what earns a “strong hire” signal.

God Class (SRP)

A class with 15+ methods spanning authentication, data access, business logic, and formatting. Refactor by identifying change drivers and extracting each into its own class with a focused interface.

Divergent Change (SRP)

You find yourself editing the same class for completely unrelated feature requests. “We need to change the tax calculation” and “we need to change the email template” both point to the same file. Refactor by splitting along change boundaries.

Shotgun Surgery (SRP, inverted)

A single logical change requires editing 10+ files. This means a single responsibility is scattered across too many classes. Consolidate the related logic into one class or module.

Switch Statements on Type (OCP)

A `switch` or `if-else` chain that selects behavior based on a type string or enum, and grows every time a new type is added. Replace with an interface and polymorphic dispatch.

Refused Bequest (LSP)

A subclass inherits methods from the parent but overrides them to do nothing or throw exceptions. The subclass is refusing its inheritance. Break the hierarchy and use composition or split the interface.

Fat Interface (ISP)

An interface with 10+ methods where most implementors only use 3-4 of them. Split into smaller, role-based interfaces that represent individual capabilities.

Dependency Magnet (DIP)

A class whose constructor creates its own dependencies using `new SomeConcreteClass()`. This hardwires the class to specific implementations and makes testing impossible without the real infrastructure. Accept dependencies as constructor parameters typed to interfaces.

Interview Relevance: Interviewers often present code with one or more of these smells and ask “what would you improve?” Having this smell-to-principle mapping internalized lets you answer immediately and precisely.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 39: SOLID Interview Practice and Common Mistakes

Practice Scenario

You are designing a reporting system for an analytics platform like Mixpanel. The system needs to generate reports in multiple formats (PDF, CSV, Excel), pull data

from multiple sources (PostgreSQL, Elasticsearch, Redis cache), and deliver reports through multiple channels (email, Slack webhook, S3 upload). The interviewer asks: "Walk me through your class design."

Pause the video and design the class structure for 90 seconds before continuing.

What the Interviewer Is Evaluating

Three independent dimensions of variation (format, data source, delivery channel) means three interface hierarchies composed together. A single `ReportGenerator` class that handles all three violates SRP (three change drivers), OCP (adding a new format requires modifying the class), and DIP (directly depending on PostgreSQL driver, Elasticsearch client, and SMTP library).

Common SOLID Mistakes in LLD Interviews

Mistake	Which Principle	Stronger Approach
One class handles validation, persistence, and notification	SRP	Separate classes per responsibility, composed by an orchestrator
Adding new behavior with if/else branches	OCP	Define interface, write new implementation class
Subclass throws UnsupportedOperationException	LSP + ISP	Split interface so subclass only implements methods it supports
12-method interface with most implementors leaving half empty	ISP	Split into 3-4 focused interfaces by capability
Business logic class creates `new PostgresConnection()`	DIP	Accept `DatabaseConnection` interface through constructor
"I used SOLID" without explaining which principle or why	All	Name the specific principle and the specific problem it solves

Interview Tip: Do not name SOLID principles unprompted unless the interviewer asks about design principles. Instead, apply them silently through your design decisions. If the interviewer probes ("why did you use an interface here?"), then name the principle. Saying "I am applying OCP here" before anyone asked can

sound rehearsed. Saying “I used an interface so that adding a new export format does not require modifying the existing code” demonstrates OCP naturally.

Interview Relevance: Interviewers care more about whether you apply SOLID than whether you can recite the definitions. Every design decision in your class diagram should implicitly follow these principles.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

SLIDE 40: Section Summary and Cheat Sheet

Concepts Covered

Principle	One-Line Summary
SRP	A class should have only one reason to change (one change driver)
OCP	Add new behavior by writing new classes, not modifying existing ones
LSP	Subtypes must honor the behavioral contract of the parent type
ISP	No class should be forced to implement methods it does not use
DIP	Business logic depends on interfaces, not concrete implementations
SRP + Microservices	Class-level SRP maps to service-level decomposition
OCP + Strategy	“Multiple types of X” maps to interface + implementations
LSP + ISP	Exception-throwing overrides signal both violations simultaneously
DIP + Testability	Interface dependencies enable mock-based unit testing

Common Interview Mistakes

Mistake	Stronger Approach
“This class has a single responsibility because it handles users”	“This class has three change drivers: authentication policy, profile rules, and notification delivery. Each should be a separate class.”
“I will add a case to the switch for the new type”	“I will write a new class implementing the interface. No existing code changes.”

Mistake	Stronger Approach
“Square extends Rectangle because a square is a rectangle”	“The behavioral contracts differ (setting height on a square changes width), so they should be sibling implementations of a Shape interface.”
“This interface has 12 methods because the system needs all of them”	“Not every implementor needs all 12. I will split into focused interfaces by capability.”
“I used new StripeClient() in my OrderService”	“OrderService accepts a PaymentProcessor interface. Stripe is injected from outside.”

Key Points to Remember

Principle	Violation Detection Test
SRP	Count the distinct change drivers for the class
OCP	Check if adding a new type requires editing existing code
LSP	Check if any subtype throws exceptions or has empty overrides
ISP	Check if any implementor leaves methods unimplemented
DIP	Check if business logic imports concrete infrastructure classes

How This Connects to Other Sections

- **Section 2 (OOP Principles):** SOLID builds directly on OOP. SRP refines encapsulation. OCP formalizes polymorphism. LSP constrains inheritance. ISP refines abstraction. DIP enforces composition.
- **Section 4 (Creational Patterns):** Factory Method is OCP applied to object creation. Builder isolates construction from representation (SRP for construction).
- **Section 5 (Structural Patterns):** Adapter enables DIP by wrapping incompatible interfaces. Decorator enables OCP by adding behavior without modification. Facade applies SRP to subsystem access.
- **Section 6 (Behavioral Patterns):** Strategy is the direct implementation of OCP. Observer decouples event producers from consumers (DIP). Chain of Responsibility applies SRP to request processing.
- **Section 7 (Pattern Selection):** The SOLID violation detection tests from this section become the signal-to-pattern mapping: “switch on type” maps to

Strategy (OCP), “fat interface” maps to ISP refactoring, “new in constructor” maps to Factory + DIP.

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn